

ARSITEKTUR FRONTEND INDEXEDDB OBL-QUEUEING

Solusi Redam Lonjakan Pooling Database & Antisipasi Latency Hari H

Sistem: Sapa Tamu.ku Ecosystem (Formulir Tamu, Kiosk, Check-in, Onsite)

Komponen Frontend: Browser Client Client-Side Storage (IndexedDB Engine)

Target Target: Mengurangi Beban Active Connection Pool Supabase (Limit: 60) via Rate-Limiting & Background Sync

Dokumen arsitektur ini dirancang sebagai spesifikasi cetak biru (blueprint) teknis untuk diimplementasikan oleh agen/pengembang ke dalam sistem frontend **Sapa Tamu.ku**. Konsep utama dari arsitektur ini adalah memposisikan **IndexedDB** pada browser sebagai benteng pertahanan (buffer memori lokal) sebelum data langsung dilempar ke Supabase/Google Apps Script.

1. Skema Basis Data Lokal (IndexedDB Schema)

IndexedDB akan mengelola satu database lokal bernama SapaTamuOfflineDB dengan struktur object store tunggal yang bertindak sebagai antrean asinkronus:

Database Name : SapaTamuOfflineDB
Version : 1
Object Store : checkin_queue
Key Path : id (autoIncrement: true)

[Index Yang Dibuat]

- status_sync : Berfungsi memfilter data yang 'pending' atau 'failed'
- timestamp : Mengurutkan antrean dari yang paling awal masuk (FIFO)

Struktur data JSON item antrean di dalam IndexedDB:

```
{
  "id": 1, // Auto-increment primary key
  "tamu_id": "UUID-12345", // ID Tamu asli
  "payload": {
    "nama": "Budi Santoso",
    "kategori": "VIP",
    "waktu_scan": "2026-07-03T14:00:00Z"
  },
  "status_sync": "pending", // Opsi status: pending | syncing | failed
  "percobaan": 0, // Jumlah retries jika terjadi gagal koneksi
  "error_log": "" // Catatan jika response server error
}
```

2. Alur Logika UI & Siklus Antrean (Data Flow Lifecycle)

Pola komunikasi data tidak lagi menembak langsung ke server secara agresif, melainkan dikendalikan melalui mekanisme tiga gerbang berikut:

1. **Aksi Scan/Input (Frontend Gate):** Ketika panitia/usher melakukan input data check-in pada halaman Kiosk/ Onsite, data langsung di-*insert* ke dalam IndexedDB dengan status pending.
2. **Respons UX Instan (Optimistic Toast):** Sistem langsung menutup modal scan dan memunculkan toast responsif berwarna kuning: "Data Diterima (Mengantre Sinkronisasi... [Kapasitas Antrean: N])". Langkah ini menjaga kelancaran alur antrean fisik tamu di lokasi.
3. **Background Worker Loop (Synchronization Engine):** Sebuah fungsi berkala (interval worker) berjalan di latar belakang setiap **1.5 hingga 2 detik sekali** untuk memproses antrean satu per satu secara terkontrol.

3. Spesifikasi Background Sync Worker (Kode Referensi)

Berikut adalah logika algoritma pemrosesan berkala (Rate-Limiter) yang wajib disuntikkan ke dalam file JavaScript utama pada halaman Kiosk, Check-in, dan Onsite:

```
// Menjalankan sinkronisasi secara berkala tiap 2000 milidetik
setInterval(async () => {
  if (navigator.onLine) {
    const nextQueueItem = await getNextPendingQueue();
    if (nextQueueItem) {
      await eksekusiKirimKeSupabase(nextQueueItem);
    }
  }
}, 2000);

async function eksekusiKirimKeSupabase(item) {
  // 1. Kunci status agar tidak dieksekusi double oleh loop lain
  await updateStatusQueue(item.id, 'syncing');

  try {
    // Menembak database dengan tempo terkontrol
    const response = await fetch('YOUR_API_ENDPOINT_URL', {
      method: 'POST',
      body: JSON.stringify(item.payload)
    });

    if (response.ok) {
      // 2. Jika sukses, hapus dari IndexedDB agar memori lokal bersih
      await hapusDariQueue(item.id);
      updateToastStatusSuccess(item.payload.nama);
    } else {
      throw new Error('Server Responded with Error Code');
    }
  } catch (error) {
    // 3. Jika gagal karena jaringan/timeout, kembalikan ke status failed untuk dicoba lagi nanti
    await updateStatusQueue(item.id, 'failed', error.message);
  }
}
```

```
}  
}
```

Rekomendasi Integrasi Agen:

Arsitektur ini dinilai **sangat cocok** untuk menjembatani keterbatasan limit pool 60 koneksi milik Supabase sebelum sistem backend kamu sepenuhnya dimigrasikan ke VPS mandiri. Arsitektur ini berfungsi sebagai peredam kejut elektrik sehingga ketukan request dari 3-4 device browser Kiosk tidak akan pernah menembak database secara bersamaan pada detik yang sama.